

MLNS Kaggle Challenge - Link Prediction in an Actor Co-occurrence Network

Team JDRX

Xavier Plantier Julian Dequatre Riccardo Castellano Daniel Fernandez de la Mela
April 2026

Overview

We are given an actor co-occurrence graph from which a random subset of edges has been removed, together with a bag-of-keywords representation of each node extracted from its Wikipedia page. We cast link prediction as a binary classification problem on pair-level feature vectors combining topological, textual and mesoscopic descriptors, and we train a gradient-boosted tree model (XGBoost) under a 10-fold stratified cross-validation scheme in which the graph is rebuilt from the training fold only, to prevent any leakage of validation edges into feature construction. Our best submission reaches a public leaderboard AUC of **0.85755**, compared to 0.84655 for a pure XGBoost without mesoscopic features and 0.82549 for a LightGCN-based pipeline. The overall pipeline is summarized in Figure 1.

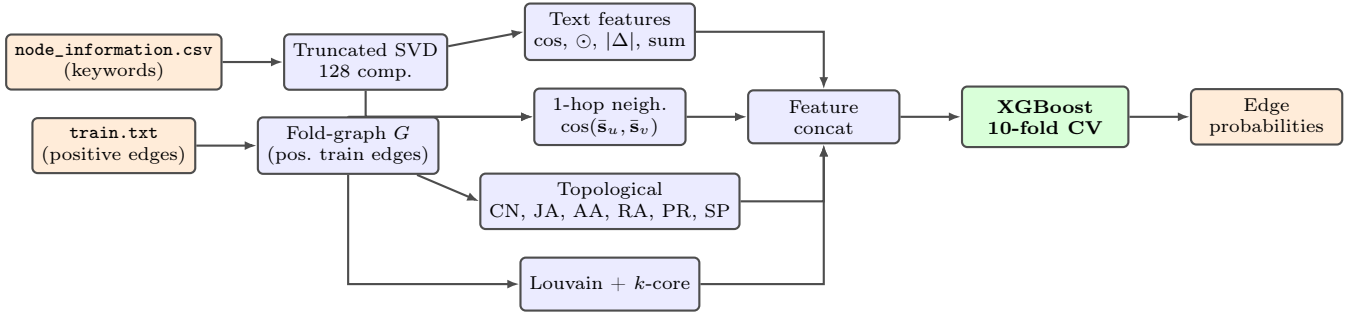


Figure 1: End-to-end pipeline. Orange = data, blue = feature blocks, green = model.

1 Feature Engineering

We group our features into four complementary families, motivated by the link-prediction literature [1]: (i) classical topological scores, (ii) textual similarity, (iii) a lightweight 1-hop neighborhood embedding, and (iv) community / k -core descriptors.

1.1 Topological features

For each candidate pair (u, v) we extract the following scores from the fold-specific graph G : *common neighbors* $|\Gamma(u) \cap \Gamma(v)|$, *Jaccard coefficient*, *Adamic-Adar* [2] $\sum_{w \in \Gamma(u) \cap \Gamma(v)} 1/\log d_w$, *Resource Allocation* $\sum 1/d_w$, *Salton* and *Sørensen* indices, degrees d_u, d_v and their product (*preferential attachment*), *shortest-path length* (capped at 10 when disconnected), the product of *PageRank* scores, and local *clustering coefficients* $c_u, c_v, c_u c_v$. Each score captures a different facet of the likelihood of a missing edge: CN / Jaccard / Sørensen quantify direct overlap, AA and RA re-weight common neighbors by how discriminative they are (a hub shared by u and v is less informative than an obscure neighbor), preferential attachment captures hub-to-hub attraction, and PageRank together with shortest-path length inject a global reachability signal that purely local scores miss.

1.2 Textual features from node_information.csv

The raw keyword vectors are kept as a dense matrix for direct similarity computation (dot product and cosine), and they are simultaneously compressed with a **Truncated SVD** of 128 components (LSA-style) to produce dense, denoised embeddings $\mathbf{s}_u \in \mathbb{R}^{128}$. From those we derive a cosine similarity, a Euclidean distance, and three *vector-valued* interactions that are fed directly to the classifier: the Hadamard product $\mathbf{s}_u \odot \mathbf{s}_v$, the absolute difference $|\mathbf{s}_u - \mathbf{s}_v|$, and the sum $\mathbf{s}_u + \mathbf{s}_v$. Hadamard and $|\cdot|$ are the standard edge operators used in **node2vec** [3] to turn node embeddings into edge features; we add the element-wise sum because with tree ensembles it provides an additional cheap non-linear interaction. This yields $3 \times 128 = 384$ dense features, allowing XGBoost to exploit per-dimension topic interactions instead of compressing everything into a single scalar similarity.

1.3 1-hop neighborhood embedding (GraphSAGE-style)

Inspired by the first aggregation layer of GraphSAGE [4], we compute for each node the mean of its neighbors' SVD embeddings, $\mathbf{n}_u = \frac{1}{|\Gamma(u)|} \sum_{w \in \Gamma(u)} \mathbf{s}_w$, and use $\cos(\mathbf{n}_u, \mathbf{n}_v)$ as a scalar feature. It measures how similar the *textual contexts* of the two actors are, independently of their own Wikipedia keywords: two actors whose co-stars

talk about the same topics are likely to be connected, even when their own keyword vectors differ. This single feature turned out to be one of the strongest non-topological signals in our ablation, and it is significantly cheaper to compute than a full-blown GNN forward pass.

1.4 Community and k -core descriptors

We run **Louvain** [6] community detection on the fold graph and add a same-community indicator, the community sizes of u and v , and the min/max ratio of these sizes. We also compute the k -core decomposition and derive k_u , k_v , $\min(k_u, k_v)$ and $|k_u - k_v|$. The intuition is that actors in the same community or in deep shells of the network are far more likely to co-occur on Wikipedia, and asymmetries in k -core levels penalize pairs mixing a periphery node with a core node.

1.5 Ablation and feature selection

XGBoost performs implicit feature selection through split gain, so we did not prune features manually; instead, we ran incremental ablations, adding one family at a time and measuring the impact on the public leaderboard. Table 1 and Figure 2 summarize the scores of our three main submissions.

Feature set	Public AUC
LightGCN embeddings + topological + SVD Hadamard (5 folds)	0.82549
Topological + SVD scalars + raw text similarity (pure XGBoost, 5 folds)	0.84655
Full pipeline: topo + SVD vectors + 1-hop neigh + Louvain + k-core (10 folds)	0.85755

Table 1: Incremental ablation on the public leaderboard.

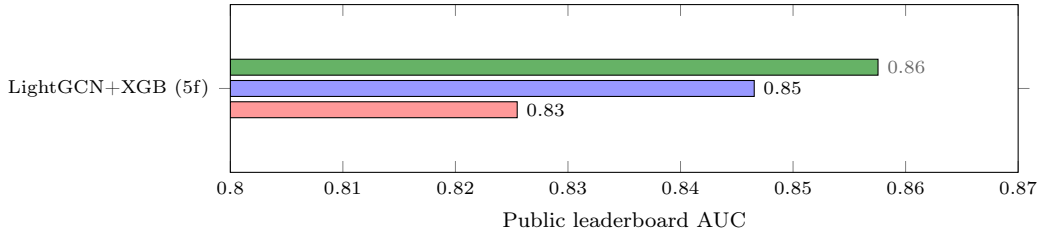


Figure 2: Visual comparison of the three submissions.

The biggest individual jumps came from Adamic-Adar and PageRank within the topological block, from the SVD Hadamard / diff / sum vectors within the textual block, and from the 1-hop neighborhood cosine. Adding Louvain and k -core features, together with moving from 5 to 10 folds, gave the final +0.011 jump that produced our best submission. We also inspected XGBoost’s gain-based importances to confirm that no single family was dominating: the top-20 split features always mixed topological scores (AA, PageRank product, shortest path), SVD Hadamard / difference dimensions, and the 1-hop neighborhood cosine.

2 Model Tuning and Comparison

2.1 Models tried

We benchmarked three very different families of models on the same data and CV protocol, so that the comparison reflects modeling choices rather than data splits:

Model	Public AUC	Comment
LightGCN + MLP link predictor [5]	0.82549	End-to-end GNN; weaker than GBDT once rich hand-crafted features are used.
XGBoost on topo + SVD + raw text (5 folds)	0.84655	Strong baseline, no community / k -core / neighborhood cosine.
XGBoost on full feature set (10 folds, final)	0.85755	Adds Louvain, k -core, 1-hop neigh. cosine; best submission.

Table 2: Model comparison under identical CV protocol.

We initially tried an end-to-end **LightGCN** [5] pipeline: a linear projection of the SVD features followed by 3 LGConv layers (weight-less neighborhood propagation averaged across layers), plugged into a 2-layer MLP link predictor trained with binary cross-entropy, and then fed back as a cosine similarity feature to an XGBoost on top of the usual topological features. Although LightGCN does capture higher-order neighborhood information, it under-performed the pure-XGBoost pipeline by more than 2 AUC points. Two factors explain this: (i) the dataset is relatively small, so a carefully chosen set of topological and textual features already saturates most of the signal a GNN could extract, and (ii) GBDT natively handles heterogeneous, unscaled, high-dimensional features (counts,

similarities, 384 SVD dimensions), whereas the GNN has to compress everything into a fixed-size embedding. We therefore selected GBDT as the final model, with LightGCN kept only as a comparison point.

2.2 XGBoost hyperparameter tuning

We tuned XGBoost through a coarse-to-fine grid search, optimizing the OOF AUC over the 10-fold stratified split rather than a single validation set, to reduce variance. The search space and the final retained configuration are summarized in Table 3.

Hyperparameter	Search space	Final value
<code>max_depth</code>	{4, 5, 6, 7}	5
<code>learning_rate</code>	{0.005, 0.01, 0.02}	0.005
<code>subsample</code>	{0.7, 0.8, 0.9}	0.8
<code>colsample_bytree</code>	{0.2, 0.25, 0.5, 0.8}	0.25
<code>gamma</code>	{0, 5, 10, 20, 23, 25}	23
<code>min_child_weight</code>	{1, 3, 5}	1
<code>reg_lambda</code>	—	1
<code>n_estimators</code>	—	10000 (early stop, 300 rounds)

Table 3: XGBoost hyperparameter search space and final configuration.

Several of these choices are explicitly anti-overfitting. The small `learning_rate` combined with large patience makes the optimization finer-grained and less prone to memorizing spurious interactions. The aggressive `colsample_bytree`= 0.25 is crucial because we have 384 correlated SVD features: forcing each tree to see only a quarter of them acts as an implicit regularizer and decorrelates the ensemble. A moderately high `gamma` penalizes shallow, noisy splits, and early stopping on the held-out fold ensures that the number of trees is chosen by validation performance rather than by a fixed budget.

For LightGCN, we used a 3-layer architecture, Adam with learning rate 10^{-2} , weight decay 10^{-3} , gradient clipping at 1, `ReduceLROnPlateau` scheduling, and early stopping on the training loss with patience 100.

2.3 Cross-validation protocol and leakage avoidance

A subtlety specific to link prediction is that *features themselves depend on the graph*: if the graph used to compute topological scores, Louvain, k -core or neighborhood embeddings already contains the validation edges, the model receives a look-ahead signal and CV scores become optimistic. To avoid this, for each of the 10 folds we apply the following leakage-free procedure:

1. **Rebuild** the graph G from the positive training edges *of that fold only*;
2. **Recompute** Louvain communities, k -core decomposition and 1-hop neighborhood embeddings on this fold-specific graph;
3. **Extract** features for the training, validation, and test pairs using *only* this graph.

The final test prediction is the average of the 10 per-fold test predictions, which also acts as an additional regularizer. Our OOF AUC stayed close to the public AUC, suggesting the model does not overfit the public leaderboard.

3 Conclusion

Our final pipeline combines classical link-prediction heuristics, SVD-based text similarity features (including vector-valued edge operators inspired by `node2vec`), a GraphSAGE-style 1-hop neighborhood embedding, and Louvain / k -core descriptors, all fed to an XGBoost classifier trained under a leakage-free 10-fold CV. The ablation confirms that each family contributes, with the SVD edge vectors, the neighborhood cosine and the mesoscopic features being the most impactful additions over a classical baseline. A full end-to-end LightGCN alternative was benchmarked but under-performed the GBDT approach by about two AUC points.

References

- [1] L. Lü and T. Zhou. *Link prediction in complex networks: A survey*. Physica A, 2011.
- [2] L. Adamic and E. Adar. *Friends and neighbors on the web*. Social Networks, 2003.
- [3] A. Grover and J. Leskovec. *node2vec: Scalable feature learning for networks*. KDD, 2016.
- [4] W. Hamilton, R. Ying and J. Leskovec. *Inductive representation learning on large graphs (GraphSAGE)*. NeurIPS, 2017.
- [5] X. He et al. *LightGCN: Simplifying and powering graph convolution network for recommendation*. SIGIR, 2020.
- [6] V. Blondel et al. *Fast unfolding of communities in large networks*. J. Stat. Mech., 2008.